EXPERIMENT 3

SYSTEM CALLS – fork(), exec(), getpid(), opendir(), readdir()

AIM: To demonstrate process creation, process identification, program execution, and directory handling using system/library calls in C.

SOFTWARE REQUIRED: Oracle VirtualBox, Fedora Linux, GCC compiler, Terminal.

PROCEDURE:

1. Start.
2. Include header files: Include stdio.h, stdlib.h and unistd.h.
3. Variable declaration: Declare an integer variable pid to store the return value of fork().
4. Create a process: Call fork() to create a child process.
5. If fork() returns -1, process creation fails.
6. If fork() returns 0, the current process is the child.
7. If fork() returns a positive value, the current process is the parent.
8. Print the first statement: Display THIS LINE EXECUTED TWICE. Both processes execute this statement.
9. Check process creation: If pid == -1, display CHILD PROCESS NOT CREATED and terminate the program.
10. Child process: If pid == 0, display the child's process ID using getpid() and its parent's process ID using getppid().
11. Parent process: If pid > 0, display the parent's process ID using getpid() and its parent process ID using getppid().
12. Print the final statement: Display IT CAN BE EXECUTED TWICE from both processes.
13. Stop.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <dirent.h>
```

```
int main(void)
{
    pid_t pid = fork();
    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }
    if (pid == 0)
    {
        printf("Child process PID: %d\n", getpid());
        printf("Executing ls using exec...\n");
        fflush(stdout);
        execlp("ls", "ls", "-1", (char *)NULL);
        perror("exec failed");
        exit(1);
    }
    else
    {
        printf("Parent process PID: %d\n", getpid());
        wait(NULL);
        DIR *dir = opendir(".");
        if (dir == NULL)
        {
            perror("opendir failed");
            return 1;
        }
        printf("\nDirectory contents:\n");
        struct dirent *entry;
        while ((entry = readdir(dir)) != NULL)
        {
            printf("%s\n", entry->d_name);
        }
        closedir(dir);
    }
    return 0;
}
```

OUTPUT:

```
[srisha@fedora ~]$ gcc system_calls.c -o system_calls
[srisha@fedora ~]$ ./system_calls
Parent process PID: 4123
Child process PID: 4124
Executing ls using exec...
Desktop
Documents
Downloads
system_calls
system_calls.c

Directory contents:
.
..
Desktop
Documents
Downloads
system_calls
system_calls.c
[srisha@fedora ~]$
```

RESULT: Thus, the fork(), execlp(), getpid(), opendir() and readdir() functions were implemented successfully to demonstrate process creation, program execution and directory handling.